Memory Arch.

Address Space

Integration
RAM / FLASH / Periph Mem.

32-bit
4 GB Space of Add

8-bit
25% Space of Add

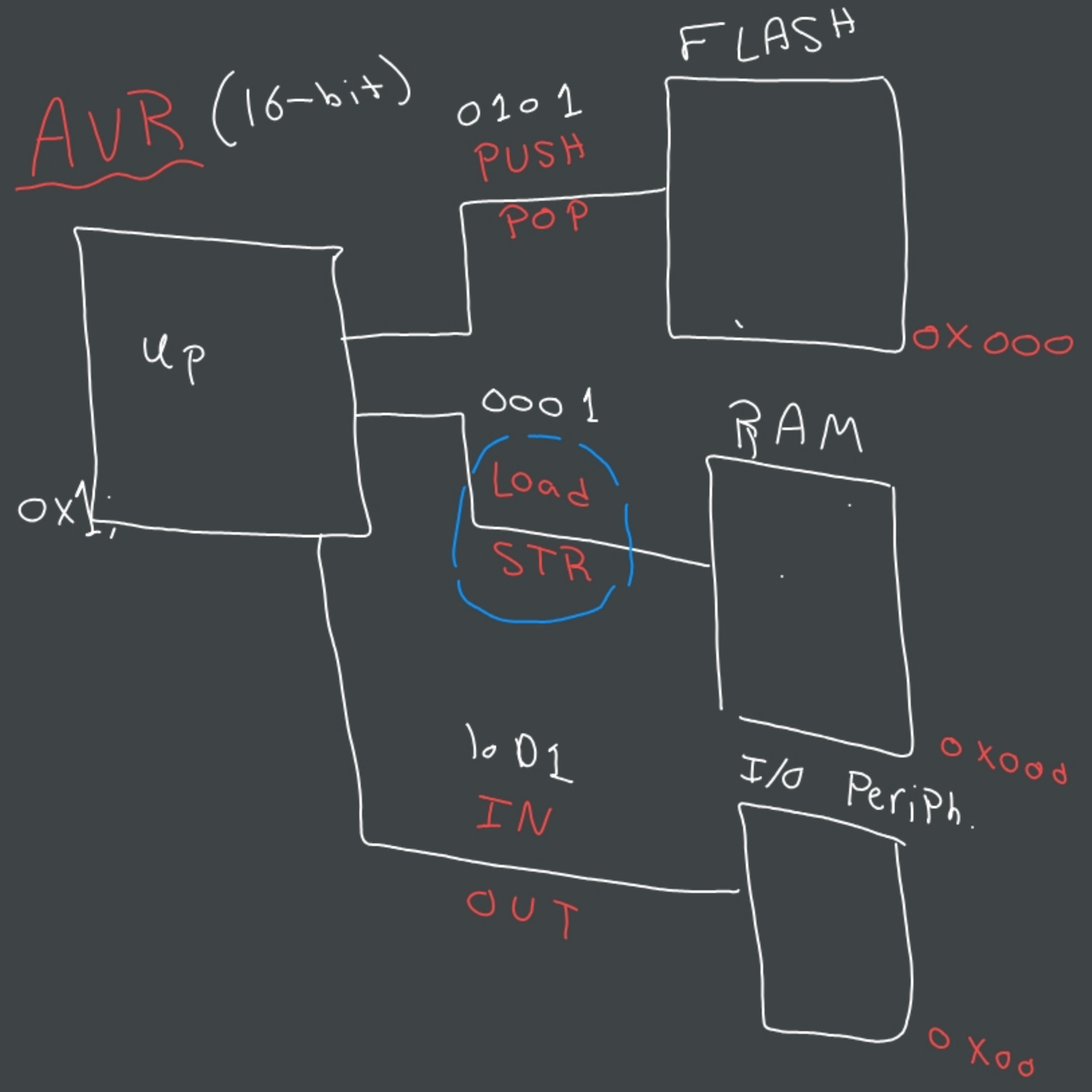
- -

*(~~u32~~ u8 *)(0x00) = 0x1;

~~Load~~ IN
~~STR~~ OUT

Harvard

AVR (16-bit)



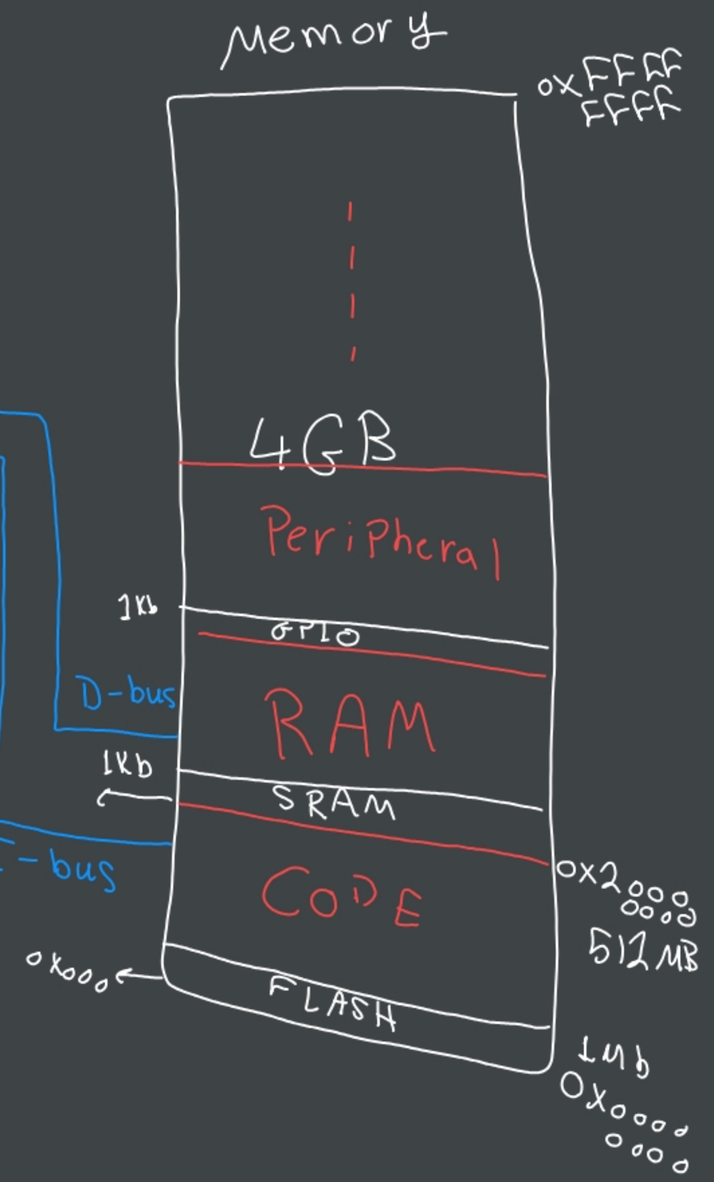
Modified Harvard Arch.

*(432*)(0x20000000) = 0x100

Up
32-bit

Port Map
↓
Harvard
↓
NOT Unique Addr.

Memory Map
↓
Modified Harvard
↓
unique Addr.



# History of ARM

1987: Acron (ARM)

Acron RISC Machine

1990: Acron ⊂ Apple ⊂ VLSI Co-op ⇝ ARM → Design
ARM → Based Licens
ARM 610

ARM → Manufactring ✗

Advanced RISC Machine

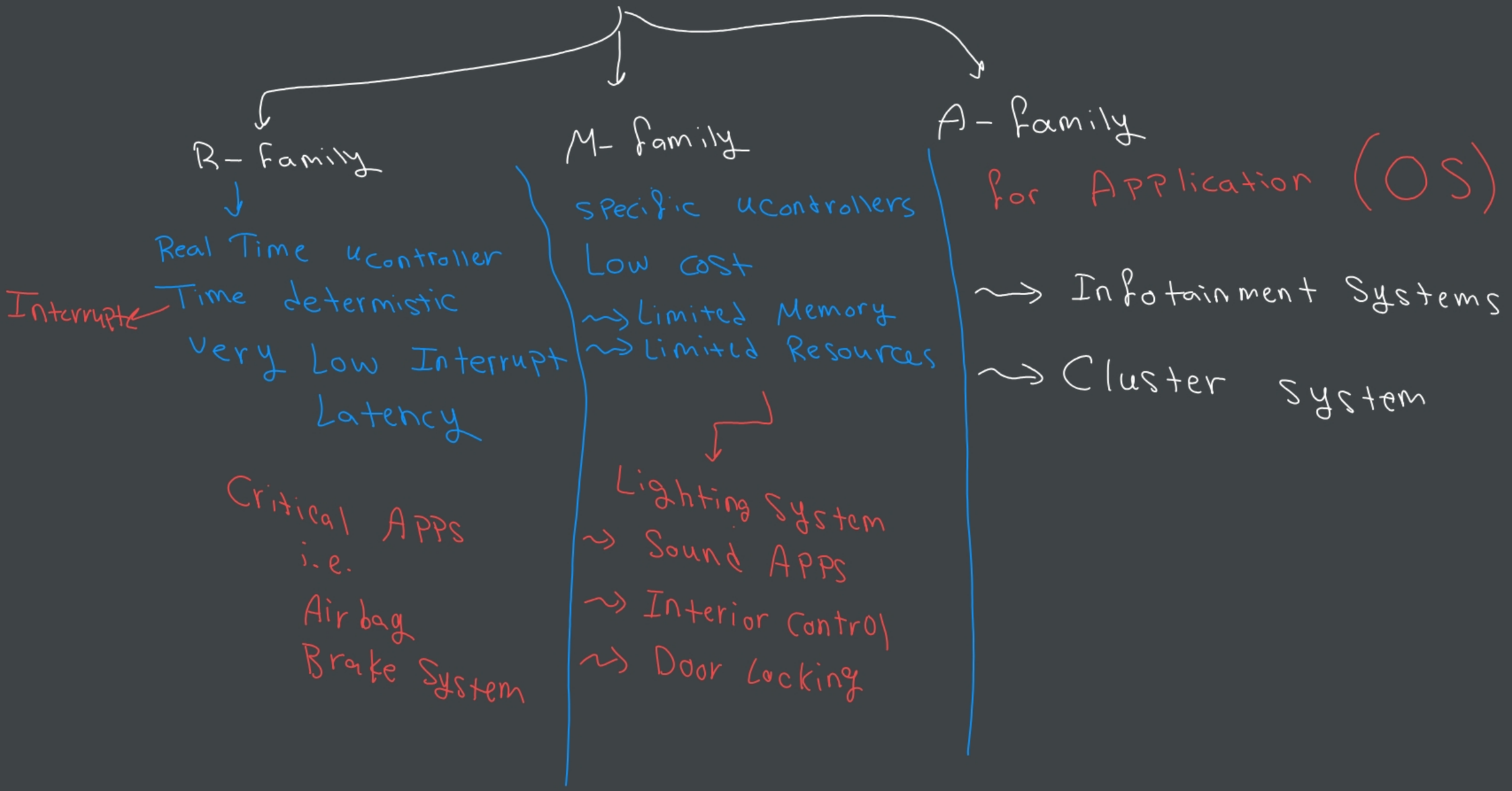
1991-93: ARM v6

⋮

2004: ARM v7 (Cortex M4)

M55

# Category of Processors

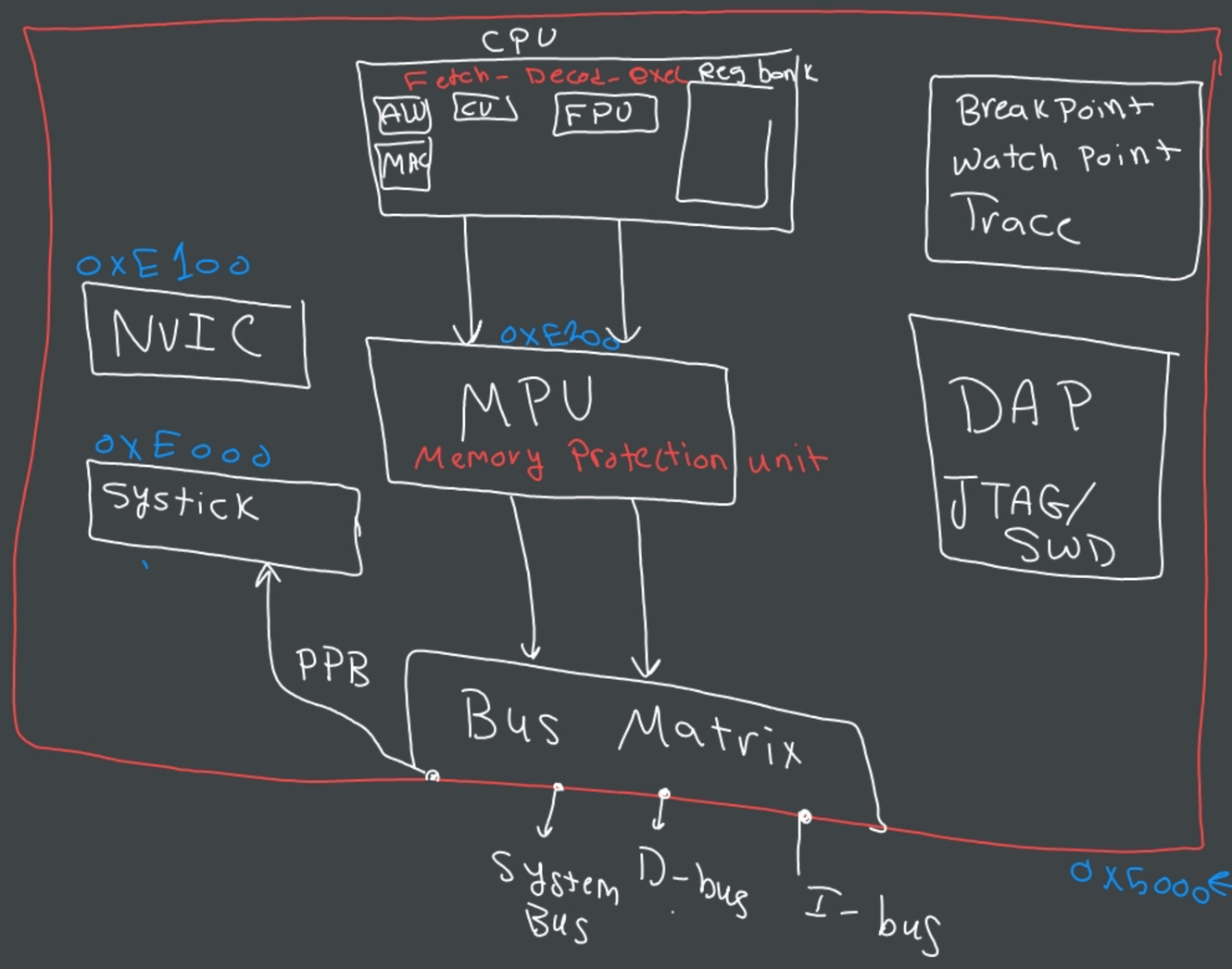


- Tri Core (AURIX) → Infinion
  - ↳ very powerful
  - Power Train Systems
- Renesus RH 850
  - ↳ Lower Cost
- ST

CORTEX M4 - F

CMSIS

Developed by ARM




Operating System

- Timer
- Interrupt Controller
- Memory Protection Unit

SW → Time1λ

float X = 1.0;



1. FPU → (Floating Point Unit)

Single Point Percion

1.1 2.5 → By Hardware

→ DSP → digital Signal Processing

1us → Calcultion

/2

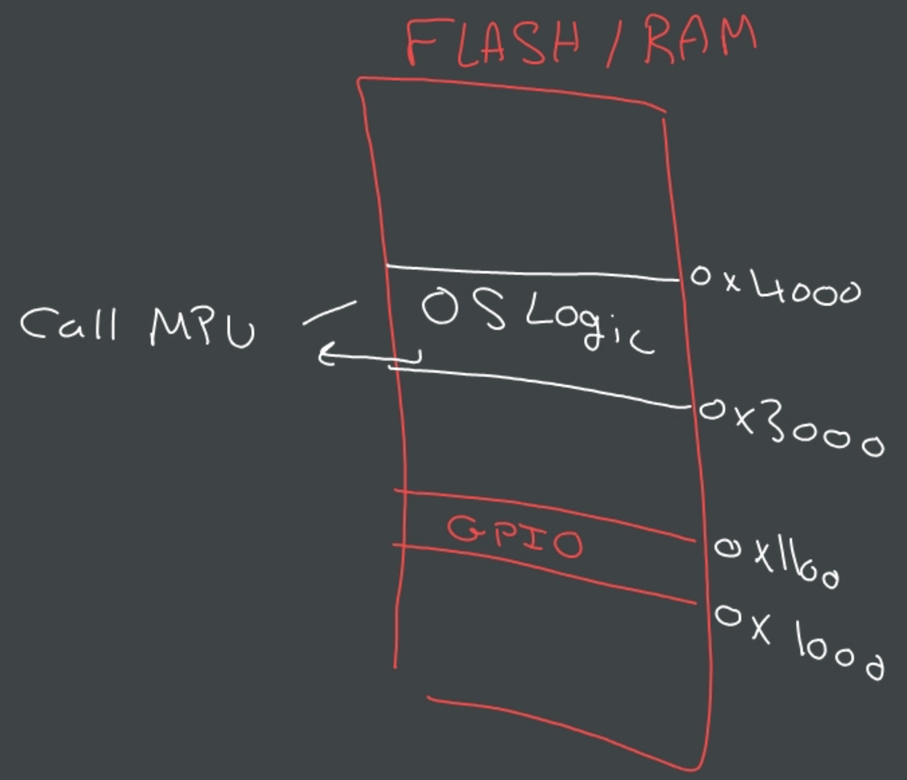
500ns → By SW

↳ 10 clk cycle

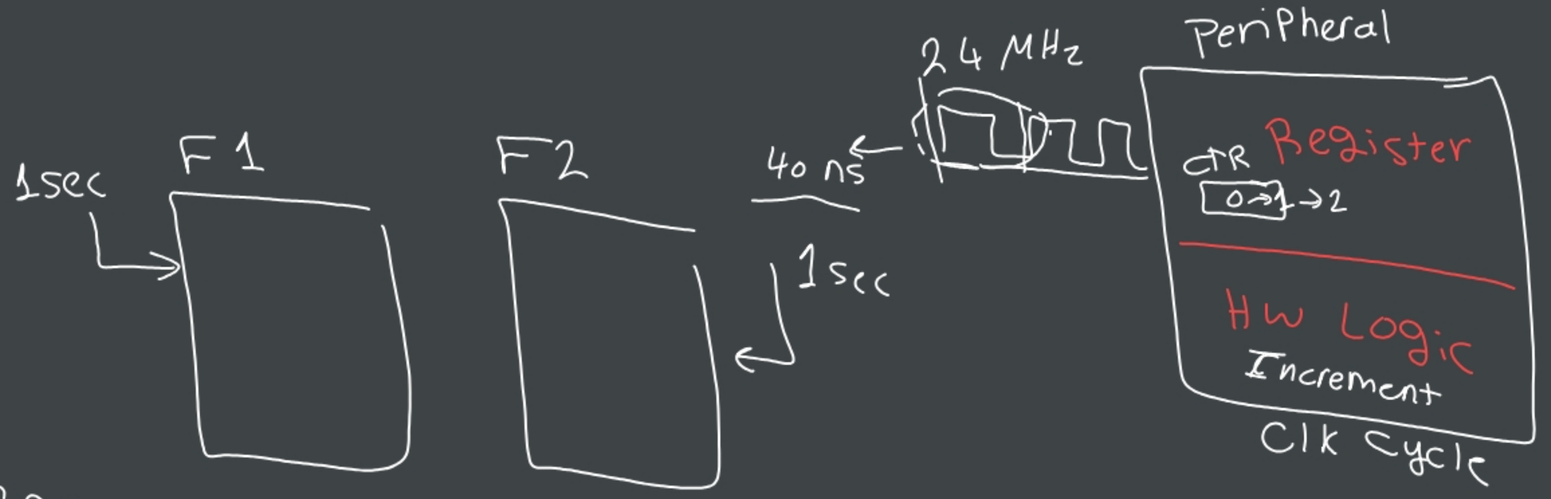
2. MPU (Memory Protection Unit).

(Optional)

↳ Ctrl / Protect Memory Accessing

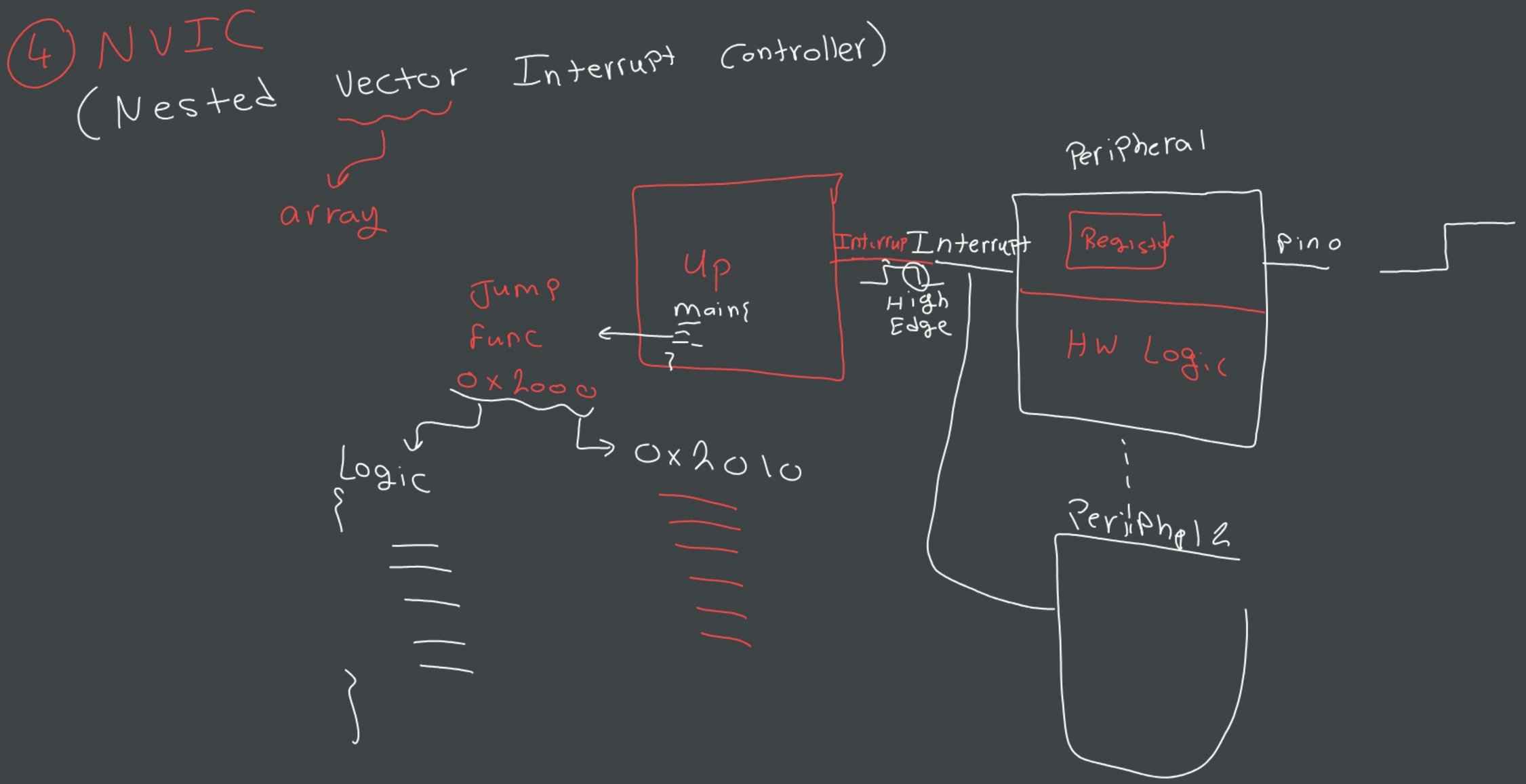


## ③ Systick Timer:




- → I/O pins GPIO
- → Adc
- → Timer
- → Interrupt Ctrl

I2C
SPI
UART
Edge detect


④ NVIC
(Nested Vector Interrupt Controller)
array
Up
main
Jump
func
0x2000
0x2010
Logic
Peripheral
Interrup
Interrupt
High
Edge
Register
HW Logic
Pino
Peripheral

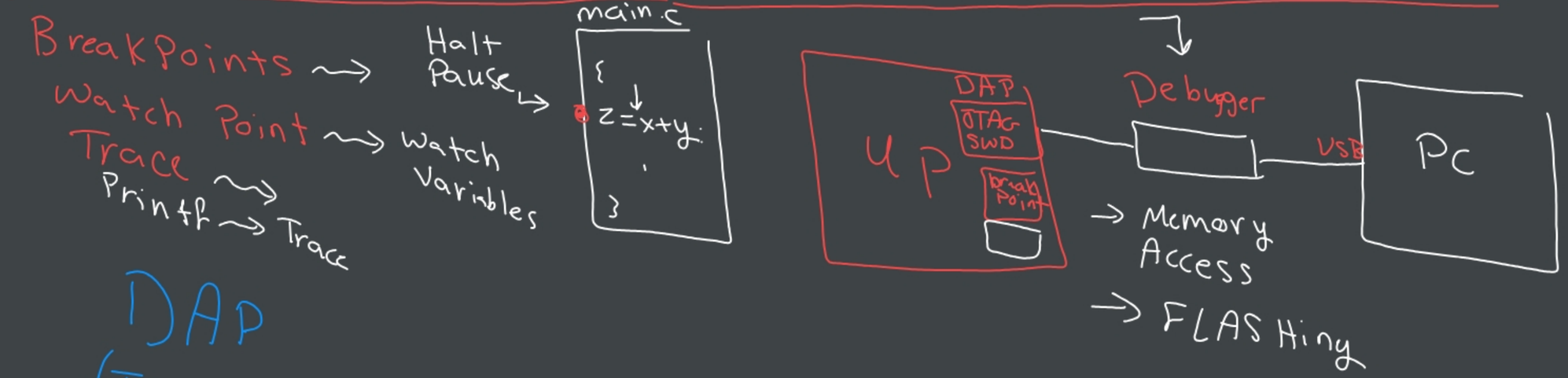
MAC: Multiplication / Accumilative Circuit

Printf

```
x = 20 * 1000;
```

1 clk cycle

Debug / Fix Issues




DAP
(JTAG - SWD)

ST-Link - Jlink - Lauterbach - UDE - —

Bus Matrix:

Routing for Buses

⇝ I-Bus: Instruction bus (AHB)

⇝ D-Bus: Data bus (AHB)

⇝ System Bus: (AHB) → (APB)

have adaptor

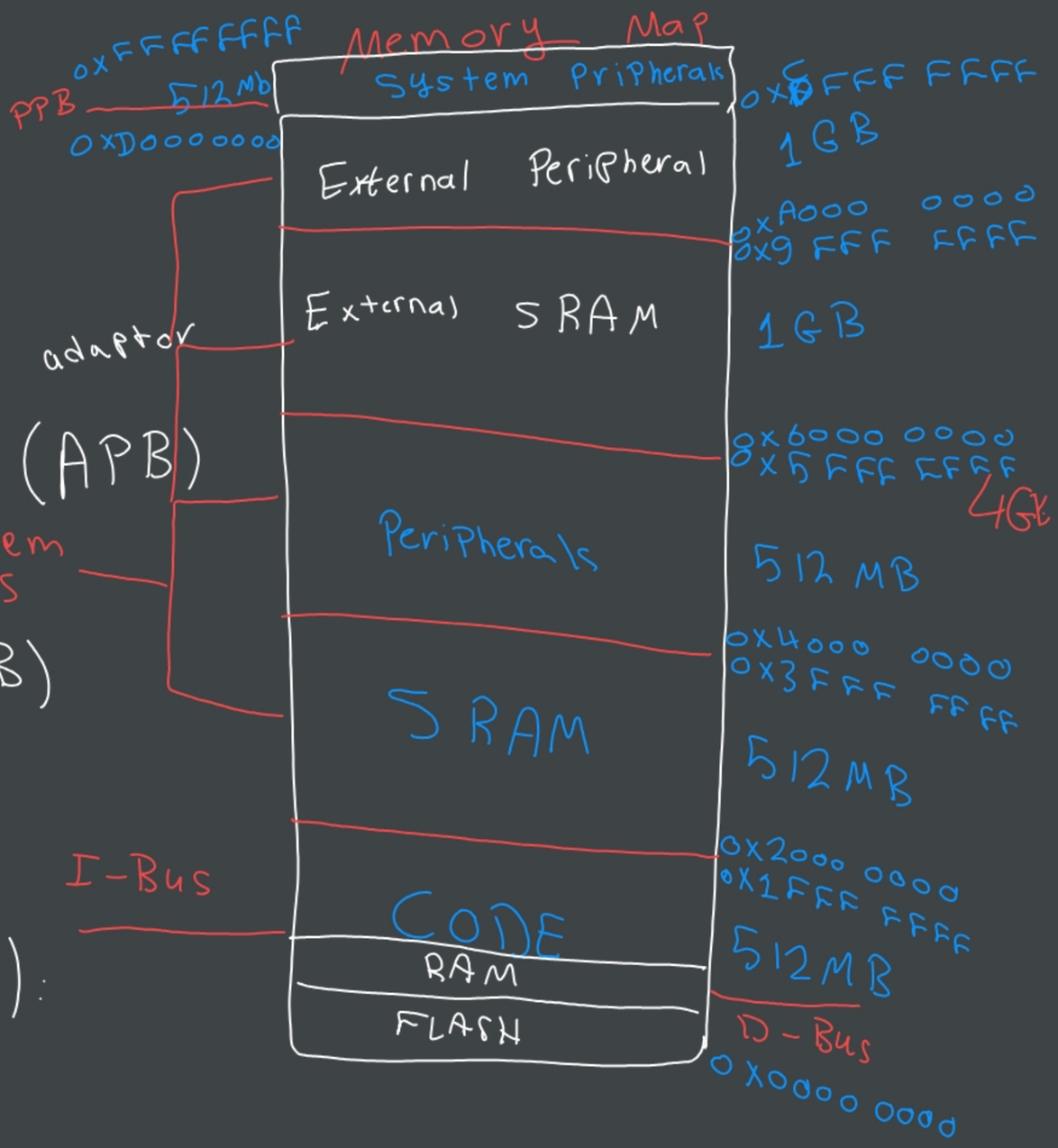
⇝ PPB: Private Peripheral Bus (AHB)

run ram

Very ←

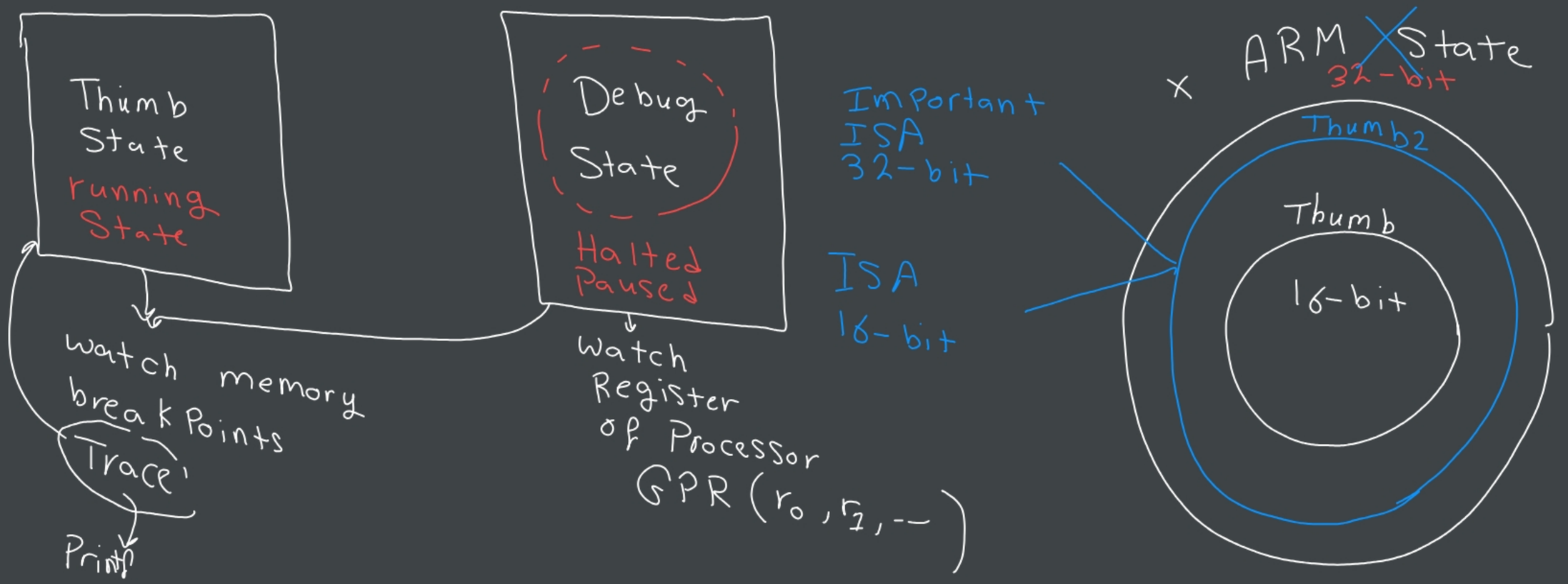
```
func1() -- attribute -- ("ram_func");
{
}
```

↳ Startup Code
↓ copy to ram

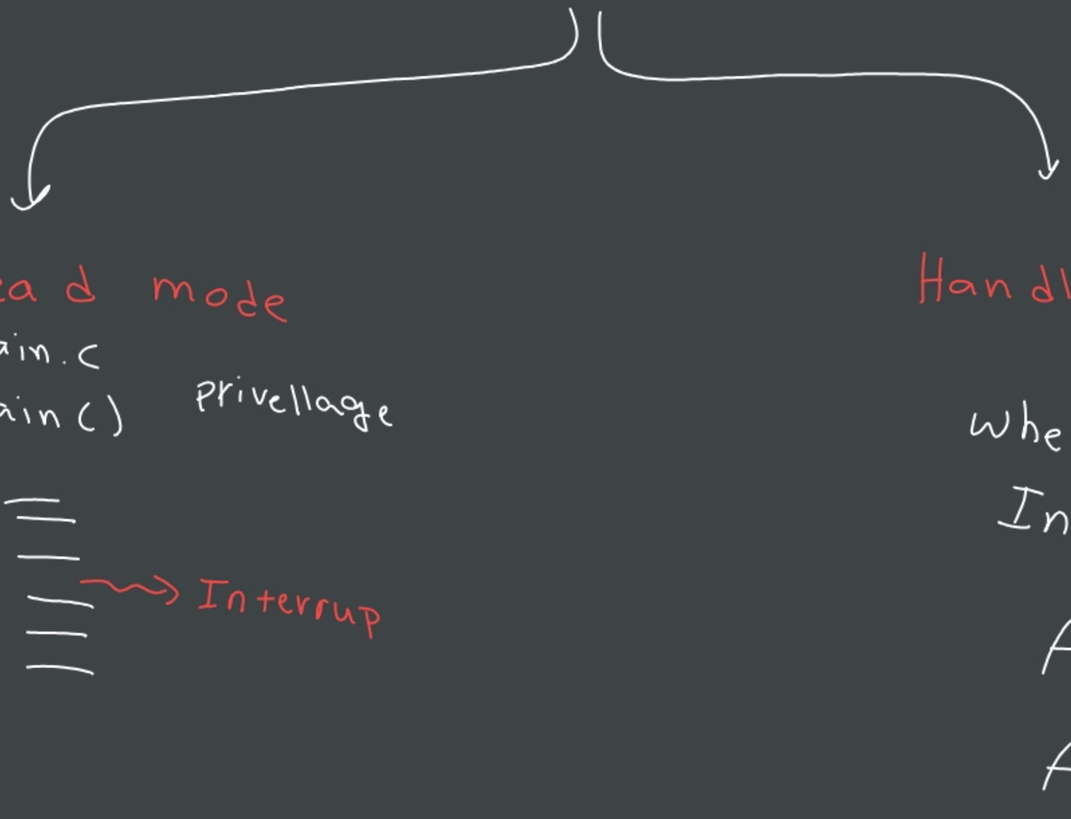


→ States ⊆ modes ⊆ Priv access

States:



modes




Handler mode

↓

when Enter

Interrupt / Exception

Always Privellage

Always MSP

access → Privellage

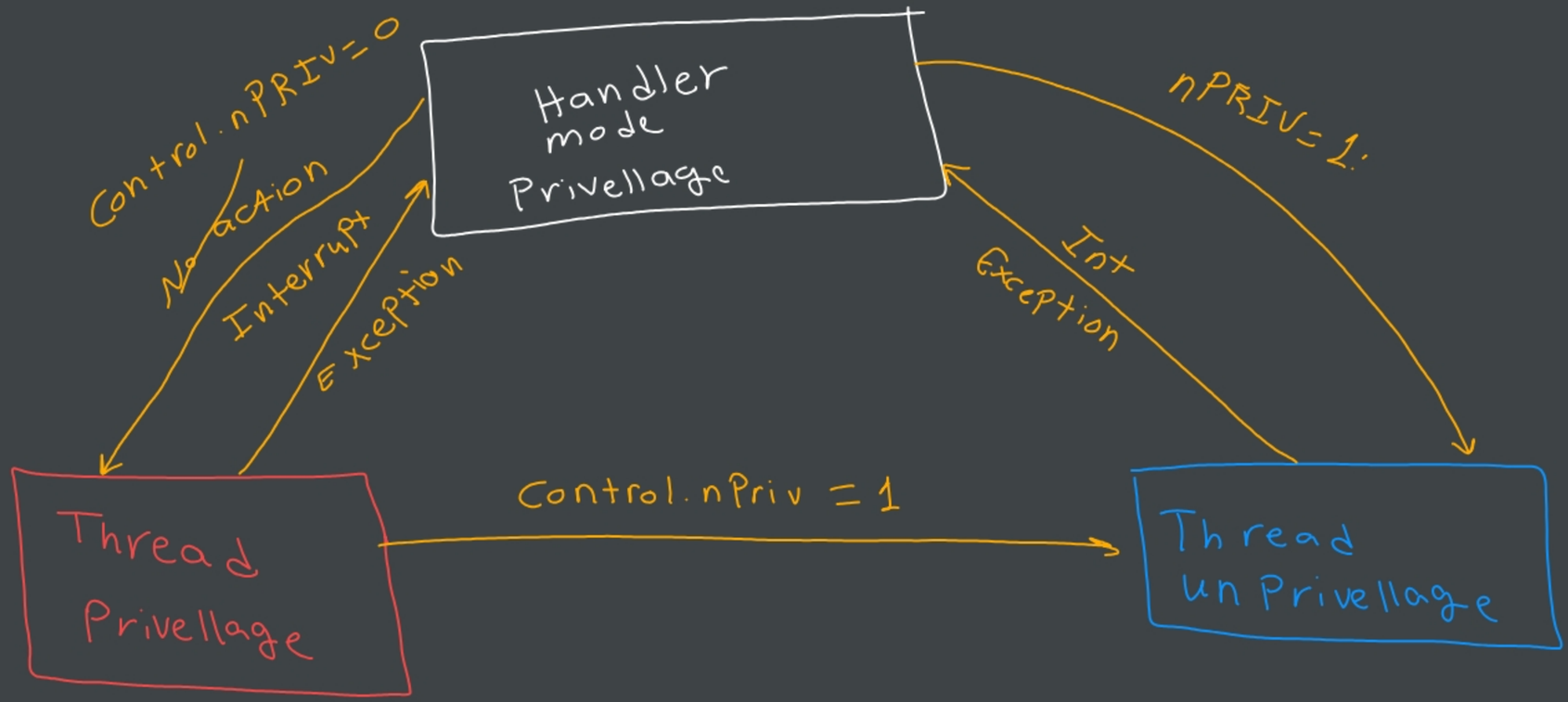
access → un Privellage

SP → MSP

SP → PSP

↙ Control.nPRIV = 1;

MSR Control ~~#~~1


Handler
mode
Privellage
Control.nPRIV=0
No action
Interrupt
Exception
nPRIV=1
Int
Exception
Control.nPriv = 1
Thread
Privellage
Thread
un Privellage

# General Purpose Registers (GPRs)

$R_0 \rightarrow R_{15}$ (32-bit)

$R_0 \rightarrow R_7$ ⇝ Thumb ISA (16-bit)

$R_8 \rightarrow R_{12}$ ⟶ 32-bit ISA

$R_0 \rightarrow R_{12}$ :⟶ To be used in APP

$R_{13}$ ⇝ SP $r_{13}$ [SP]

$R_{14}$ ⇝ LR

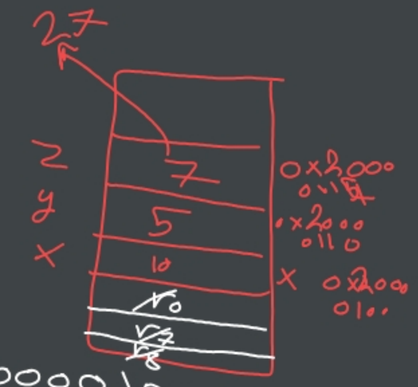
$R_{15}$ ⇝ PC

$R_0 = 0x2000\ 0100$

$R_7 = 10$ ~~10~~ 20

$R_8 = 5$ ~~5~~ 7

$z = x + y$;




$R_0 = \&x$; <

$R_7 = *(u32*)(0x2000\ 0100)$

Branch with Link

Func2();

↳ c = a+b; x

BL Func2

0x2000 0100 ← Func1

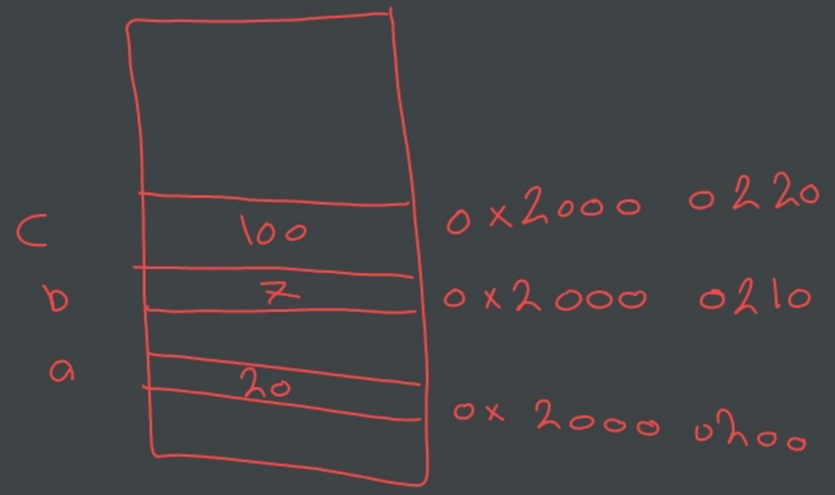
```
MOV  R0, #0x20000100
LDR  R0, =0x2000 0100
LDR  R7, [ R0 ]
LDR  R1, =0x2000 0110
LDR  R8, [ R1 ]
LDR  R2, =0x2000 0114
ADD  R9, R7, R8
STR  [R2], R9
```

Save Register

Restore

Func 2:

```
MOV   R0, #0x2000 0200
LDR   R7, [R0]
MOV   R0, #0x2000 0210
LDR   R8, [R0]
MOV   R0, #0x2000 0220
ADD   R9, R7, R8
STR   [R0], R9
```




$R_9 \longrightarrow C = 27$

$R_7 = 20$

$R_8 = 7$

$R_9 = 27$

$R_0 = 0x2000\ 0220$

R13 [SP]

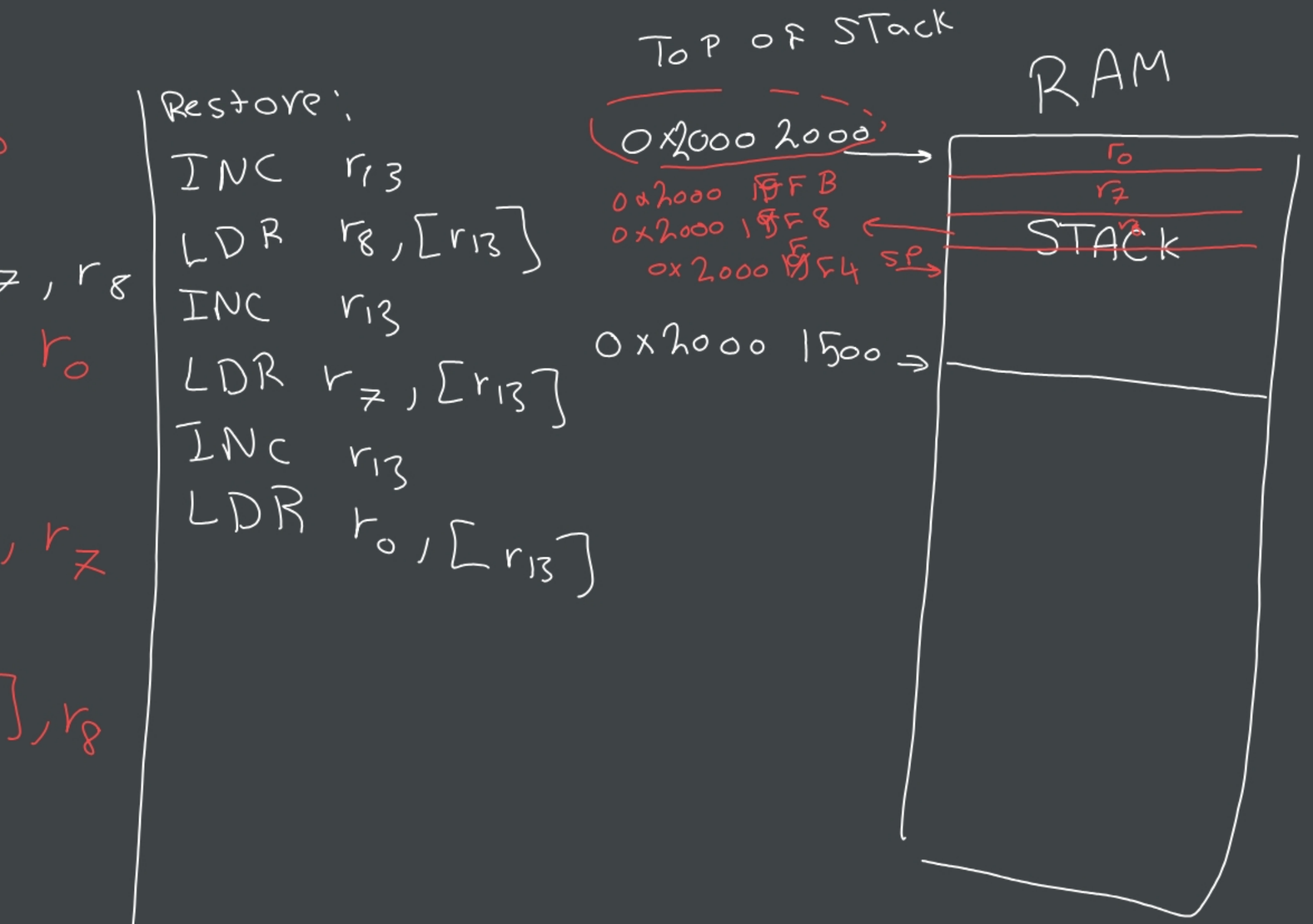
↳ Top stack
= 0x2000 2000

↳ Save $r_0, r_7, r_8$

```
STR [r13], r0
DEC r13
STR [r13], r7
DEC r13
STR [r13], r8
DEC r13
```

Restore:

```
INC r13
LDR r8, [r13]
INC r13
LDR r7, [r13]
INC r13
LDR r0, [r13]
```



→ PUSH ⇝ Save, DEC

→ POP ⇝ INC, LDR (restore)

PUSH { $r_0$ - $r_7$, $r_{12}$ } ⇝ In order from smaller → — Incremental

POP { $r_0$ - $r_3$, $r_{12}$ }

Push $r_0$

u32 func ();

$r_0$ - $r_3$ ⇝ for return

AAPCS

Advanced ARM Procedure call Standard

Caller
- Save $r_0$ - $r_3$ if needed
- $r_{12}$ → mandatory

Callee
- Save $r_4$ - $r_{11}$ if needed
- $r_{13}$
- $r_{14}$
- $r_{15}$

$r_{14}$ (Link register)

- Save return address
- Save Flags for Interrupt




Bx ⇝ return on return address

↓

Branch exist

$r_{15}$ (PC)

↳ address of next Instruction

$S_0 \rightarrow S_{31}$

↳ register for FPU

(*) Special (function) registers

↳ Control → Priv.

↳ SPSEL → MSP / PSP

↳ xPSR (Program Status register) 32-bit

xAPSR / xEPSR / xIPSR

[ -b ] [ 4-bit ] [ 3-bit ]

-b → Status Flag
- Carry
- Negative
- GE

4-bit → Current Exception running

3-bit → Thumb State

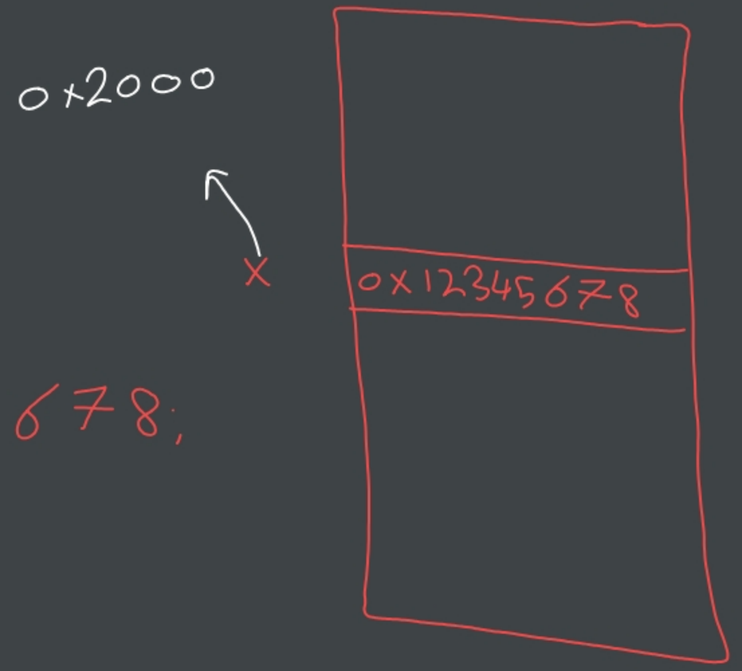
register for Interrupt

- PRIMASK
- BASEPRI
- FAULTMASK

Endianity

Little – Endian

Big Endian




```
432  x = 0x12345678;
y = *((416*)(0x2000))
Printf("%x", y);
```

Big Endian → 0x1234

Little Endian → 0x5678

C Code:

0 ← x |= 1; ⇝ x = x | 1;

0b0000 0001

Instruction Write

① ⇝ Read (ro) ⇝ Int

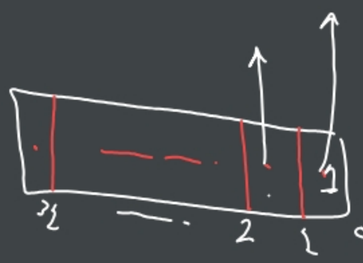
② ⇝ Modify ⇝ Interrupt

③ ⇝ Write

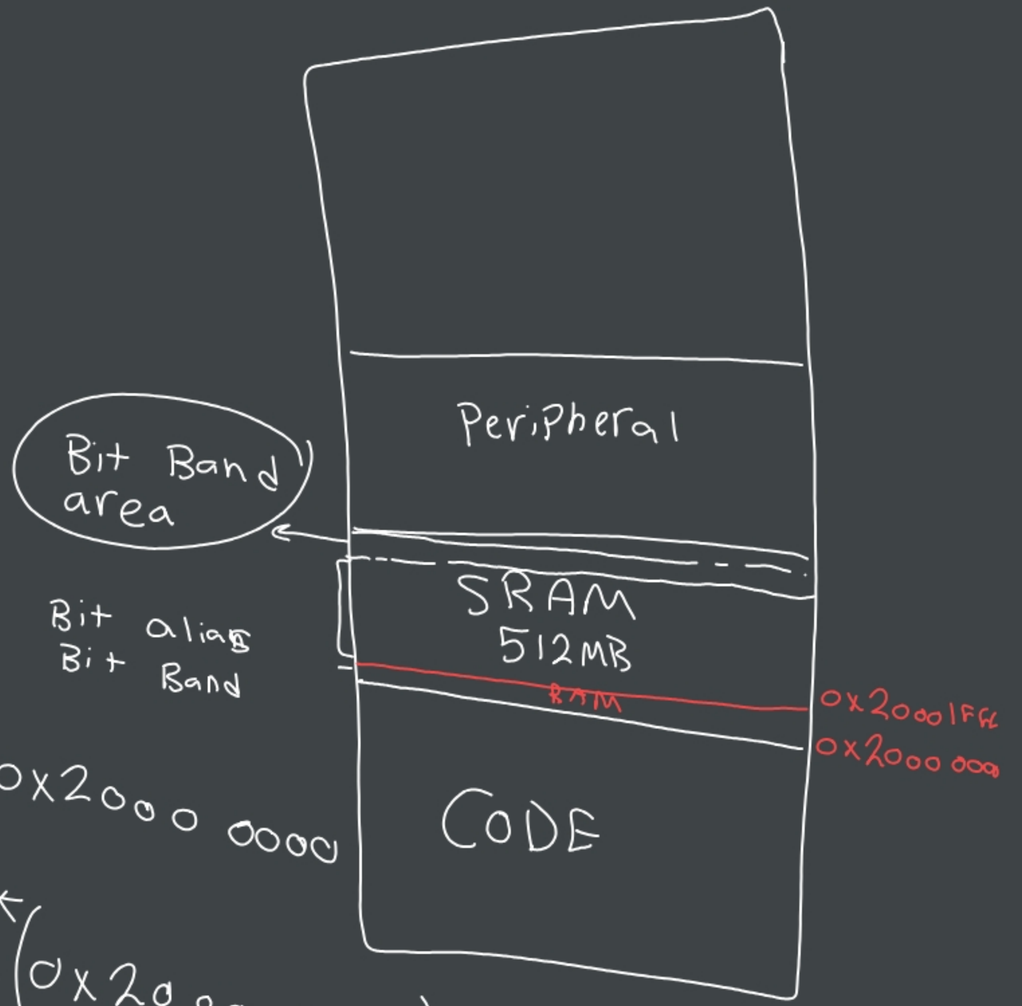
① ⇝ atomic access

② Disable Int /En Int

+




Bit alias
Bit Band




0x2000 0000

Bit band area = *(0x2000 xxxx) = 1;